

Programmation Orientée Objet pour le Calcul Scientifique : Part I Survival kit

Organisation du cours

Sous forme d'**Item**, se raccrochant à quatre catégories :

- **Outils et Langage (Tool)**
- Orienté objet (OO)
- Gestion de la mémoire (Mem)
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...
et les LLM ! Mais pas pour coder à votre place...!!

Item 1 : C++ , un langage compilé

Un point de départ : la fonction `main`

```
int main() {  
    return 0;  
} // Votre premier programme C++ prog.cpp (ou .cxx, .cc, .c++)
```

Prototype de fonction :

```
type_retour nom_ma_fonction(arguments_ou_pas){  
    corps de la fonction;  
}
```

Item 1 : C++ , un langage compilé

Comment compiler ? 1/3

- Pour démarrer : en ligne !
 - <http://coliru.stacked-crooked.com/>
 - <https://wandbox.org/>
 - <https://godbolt.org/>

Exo 1.1 : hello C++ world compilateur en ligne

Item 1 : C++ , un langage compilé

Comment compiler ? 2/3

- Ensuite avec un compilateur...Quatre principaux compilateurs :
 - GCC (Communauté, https://fr.wikipedia.org/wiki/GNU_Compiler_Collection)
 - Clang (Communauté, <https://fr.wikipedia.org/wiki/Clang>)
 - MSVC (Microsoft, https://fr.wikipedia.org/wiki/Microsoft_Visual_Studio)
 - ICPC (Intel, https://en.wikipedia.org/wiki/Intel_C%2B%2B_Compiler)

Item 1 : C++ , un langage compilé

Comment compiler ? 3/3

- sous Linux, gcc et clang facilement disponibles
- sous Windows msvc et clang accessible via [Visual Studio Community](#)
- sous MacOS, clang accessible via [XCode](#)

Commande pour compiler (gcc, clang)

```
> g++ hello_world.cpp -o hello_world.exe // gcc
> clang++ hello_world.cpp -o hello_world.exe // clang
> ./hello_world.exe
Hello C++ world
```

Exo 1.2 avec votre OS et compilateur favori, refaire le hello world.

Item 1 : C++ , un langage compilé

Quelles options de compilation de base ?

- `-o exe_name` : positionne le nom de l'exécutable
- `-Wall` : active de nombreux warnings (obligé !) voire `-Wextra`
- `-std=c++20` , `-std=c++17` , `-std=c++14` , `-std=c++11` : choix du standard C++
- `-g` : export des symboles pour le debug
- `-O2` , `-O3` : option d'optimisation

Ces options peuvent varier en fonction du compilateur (ex MSVC...)

Item 1 : C++ , un langage compilé

Deux types de fichiers existent en C++ :

- les entêtes (headers) : `Filename.h` qui sont inclus
- les sources : `Filename.cpp` qui sont compilés

```
#include <iostream> // fichier "système"  
#include "Filename.h" // fichier utilisateur
```

Les fichiers `.h` permettent de mutualiser du code.

Ils rendent accessibles le contrat de la fonction ou de la classe.

A l'édition des liens, le compilateur doit trouver l'implémentation de ce contrat.

- **Exo 1.3** reprendre le cas hello world : sans `include iostream` mais en incluant un fichier `.h` que vous aurez écrit.

Item 1 : C++, un langage compilé

Les **IDE** ou Environnement de Développement Intégré pour gagner en productivité

- gère l'édition du code (coloration syntaxique, complétion, navigation, refactoring)
- gère le système de version (git, svn,...)
- gère la compilation du code
- peut également gérer l'exécution et le debugging
- propose un agent IA...

Item 1 : C++ , un langage compilé

IDE C++ les plus connues :

- Visual Studio, Visual Studio Code (Microsoft)
- XCode (Apple)
- CLion (JetBrains)
- NetBeans (Communauté Apache Software Foundation)
- Eclipse CDT (Communauté Eclipse Foundation)

s

Item 2 : Types de bases et structure de contrôle C++

Les types de bases proches du C, <https://en.cppreference.com/w/cpp/language/types>

- entier : `int`
 - `short` / `long int`, `signed` / `unsigned`
- réel : `float`, `double`, `long double`
- caractère : `char` (`char8_t`, `char16_t`, `char32_t`)
- booléen : `bool`

Item 2 : Types de bases et structure de contrôle C++

Tailles et plage de valeurs (architecture 64 bits) 1/2

<i>Types</i>	Taille (bits)	Format	étendue
char	8	signed unsigned	−128...127 0...255
char16_t	16	signed unsigned	$\approx \pm 3.10^4$ $\approx 6.10^4$
int	32	signed unsigned	$\approx \pm 2.10^9$ $\approx 4.10^9$

Item 2 : Types de bases et structure de contrôle C++

Tailles et plage de valeurs (architecture 64 bits) 2/2

<i>Types</i>	Taille (bits)	Format	étendue
<code>long long int</code>	64	signed unsigned	$\approx \pm 9.10^{18}$ $\approx 2.10^{19}$
<code>float</code>	32	IEE754	$\approx \pm 3.10^{38}$
<code>double</code>	64	IEE754	$\approx \pm 2.10^{308}$

Pour ne pas se perdre utilisez les types STL : `std::int32_t` , `std::int64_t` ...

- <https://en.cppreference.com/w/cpp/types/integer>

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 1/6

- boucle `for (<init>;<condition>;<increment>) {<bloc>}` , identique au C

```
for (int i = 0 ; i < nb_iterations; i++) {  
    std::cout << "iteration i = " << i << "\n";  
}
```

- structure conditionnelle `if (<condition>) {<bloc>}`

```
if (condition_is_true) {  
    std::cout << "My condition is true " << std::endl;  
}
```

Exo 2.1 Afficher les entiers de 0 à 100. Puis uniquement les nombres pairs.

- Indice : l'opérateur `%` retourne le reste de la division entière...

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 2/6

- structure conditionnelle `if(<init>; <condition>){<bloc>}` avec initialisation

```
if (bool condition = (n < max); condition) {  
    std::cout << "My condition is true " << std::endl;  
}
```

Démo 2.1 : utilisation de l'initialisation dans la boucle dans l'Exo 2.1.

- structure conditionnelle `if(<condition>){<bloc>} else {<bloc>}`

```
if (n < max) {  
    std::cout << "n < max " << std::endl;  
} else {  
    std::cout << "n >= max " << std::endl;  
}
```

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 3/6

- structure conditionnelle `if(<condition>){<bloc>} else if(<condition>) {<bloc>}`

```
if (i == 0) {  
    i = 101;  
} else if (i == 1) {  
    i = 11;  
} else if (i == 2) {  
    i = 1;  
} else {  
    i++;  
}
```

Exo 2.1 (suite) : Afficher si un nombre est un multiple de 5 ou s'il est un multiple de 2 ou s'il n'est aucun des deux.

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 4/6

- structure tant que `while`

```
while (condition_is_true) {  
    std::cout << "still doing an operation." << std::endl;  
}
```

- structure faire tant que : `do <bloc> while <condition>`

```
int i = 0;  
do {  
    i++;  
} while (i < 5);
```

Exo 2.2 : écrire de deux façon différente (`while` et `do-while`) les 10 premiers entiers.

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 5/6 : compléments

- Si une `condition` est définie comme l'union de plusieurs opérations, arrêt à la première évaluation fausse

```
int z,i = 0;  
if (z > 0 && i ==3){...} // expression (i == 3 ) n'est pas évaluée
```

- Il est possible pour toutes les structures de contrôle d'omettre le scope {},
 - si une seule ligne d'instruction

```
if (i > 5) std::cout << i << std::endl;  
int a = 0;  
for (int i = 0 ; i < 5 ; ++i) a+=i;  
// Attention peut être source d'erreur, à éviter...
```

Item 2 : Types de bases et structures de contrôle C++

Les structures de contrôle 6/6 : compléments

- deux instructions pour les boucles (`for` , `while` , `do while`) :
 - `break` : sortie de la boucle
 - `continue` : sortie de l'itération courante

```
while (true) {  
    break; // sortie d'une boucle qui serait infini  
}  
for (int i = 0; i < 10; i++) {  
    if (i == 5) continue; // l'itération 5 est sautée  
    ...  
}
```

- l'instruction de contrôle `switch` sera vu dans l'item 5, associée aux `enum class` .

Item 3 : Créez et manipulez vos variables

Initialisation : tous les coups sont permis `()` , `{}` , `=`

```
bool a = true;  
int b(1);  
float c{3.14}; // initialisation uniforme recommandée ; plus robuste
```

Un seul mot d'ordre : **initialisez toujours vos variables**. Une variable non initialisée est un bug potentiel.

Item 3 : Créez et manipulez vos variables

Les opérateurs en C++ du simple au double: `+ -= < > & | , ++ == -- < < > > && ||`

```
int i = 1 + 2 ;
bool true_exp = i <= 3;
bool false_exp = i == 1;

i++; // j = i; i = i + 1; return j;
++i; // i = i + 1; return i;

false_exp = true && false; // ET logique, ne pas confondre avec & (bit à bit)
true_exp = true || false; // OU logique, ne pas confondre avec | (bit à bit)

#include <iostream>
std::cout << "Voici l'opérateur de flux" << std::endl;
int valeur_saisie_au_clavier;
std::cin >> valeur_saisie_au_clavier;
```

Item 3 : Créez et manipulez vos variables

Les opérateurs en C++ variations autour de = : +=, -=, *=, /=, %=, %, !=

```
int i = 1 ;  
i+=2 ; // i = i + 2  
i-=1;  // i = i - 1  
i*=3;  // i = i * 3  
i/=2;  // i = i / 2  
i%=2;  // i = i %2; => % donne le reste de la division entière  
  
bool true_exp = 2 != 3; // NON EGAL, opérateur logique
```

Item 3 : Créez et manipulez vos variables

L'opérateur ternaire : `(<condition>) ? true_case : false_case`

```
int i = 1 ;  
(i % 2 == 0) ? std::cout << "even" : std::cout << "odd";
```

L'opérateur virgule : `<instruction 1> , <instruction 2>`

exécute les instructions et retourne le résultat de la dernière

```
int n = 1;  
int m = (++n, n+=2, 2 * n);  
std::cout << "m = " << (++m, m) << '\n';
```

Exo 3.1 : combien vaut m ?

Item 3 : Créez et manipulez vos variables

Ordre des opérations, ne perdez pas le fil...

```
int i = 1 + 2 + 3 + 4;  
int j = 1 + 2 * 3 + 4;  
  
bool b = true && false && true;  
bool c = true || false && true;  
  
int k;  
i = j = k = 0;
```

Exo 3.2 : prévoir l'ordre des opérations ci-dessus.

Item 3 : Créez et manipulez vos variables

Ordre des opérations, ne perdez pas le fil...

```
// opérateurs arithmétiques et logiques : gauche à droite en respectant priorités
int i = 1 + 2 + 3 + 4; // ((1+2)+3)+4)
int j = 1 + 2 * 3 + 4; // ((1+(2*3))+4)

bool b = true && false && true; //((true && false) && true)
bool c = true || false && true; // (true || (false && true ))

// opérateurs d'affectation droite à gauche
int k;
i = j = k = 0; // (i = (j = (k = 0)))
```

Tous les détails : https://en.cppreference.com/w/cpp/language/operator_precedence

Item 4 : Premiers pas avec les fonctions

Les fonctions 1/8 :

- prennent des arguments en entrée ou entrée/sortie, ou non
- produisent une sortie (mot clé `return`), ou non
- s'écrivent avec le prototype suivant :

```
<out> fonction(<in_or_out_arguments...>) {<instruction bloc>}
```

```
// No in nor out
void hello_world() {std::cout << "Hello function world";}
// No out, int in
void func(int i) {std::cout << "Value is " << i;}
// int out, two int in
int func2(int i, int j) { return i+j; }
```

Item 4 : Premiers pas avec les fonctions

Les fonctions 2/8 :

- sont appelées avec leurs arguments par la syntaxe :

```
func(<in_or_out_arguments...>);
```

```
int main() {  
    hello_world(); // No in nor out  
    func(1); // No out, int in  
    int result = func2(1,2); // int out, two int in  
}
```

Exo 4.1 Ecrivez une fonction retournant la somme de trois flottants et appelez la dans un programme principal.

Item 4 : Premiers pas avec les fonctions

Les fonctions 3/8 :

- peuvent être définies en deux blocs distincts :
 - la déclaration = le prototype de la fonction

```
void hello_world();
```

- la définition = le corps de la fonction (on rappelle le prototype)

```
void hello_world() {std::cout << "Hello function world";}
```

Permet par exemple d'**exposer le prototype** de la fonction dans un fichier `.h`
sans **exposer l'implémentation** de la fonction 'cachée' dans un fichier `cpp`

Item 4 : Premiers pas avec les fonctions

Les fonctions 4/8 :

- peuvent être définies en deux blocs distincts :
 - la déclaration = le prototype de la fonction

```
void hello_world();
```

- la définition = le corps de la fonction (on rappelle le prototype)

```
void hello_world() {std::cout << "Hello function world";}
```

Exo 4.2 Séparer la déclaration de la définition de votre fonction de l'Exo 4.1. Essayez de l'appeler avec uniquement la déclaration.

Item 4 : Premiers pas avec les fonctions

Les fonctions 5/8 :

- peuvent avoir des valeurs par défaut
 - doivent être placés à la fin de la liste des arguments
 - le ou les arguments peuvent alors être omis lors de l'appel à la fonction

```
int func2(int i, int j = 0);  
int func2(int i, int j) { return i+j; }  
int main() {  
    int res = func2(1); // i =1, j = 0  
}
```

Attention la valeur par défaut ne doit être **précisée qu'une seule fois** :
dans la déclaration ou la définition

Item 4 : Premiers pas avec les fonctions

Les fonctions 6/8 :

- peuvent avoir des valeurs par défaut
 - doivent être placés à la fin de la liste des arguments
 - le ou les arguments peuvent alors être omis lors de l'appel à la fonction

```
int func2(int i, int j = 0);  
int func2(int i, int j) { return i+j; }  
int main() {  
    int res = func2(1); // i =1, j = 0  
}
```

Exo 4.3 Ajouter une valeur par défaut dans votre fonction des Exo 4.1 et Exo 4.2.

Item 4 : Premiers pas avec les fonctions

Les fonctions 7/8 :

- peuvent appeler d'autres fonctions

```
void hello_world() {std::cout << "Hello function world";}

int func2(int i, int j) {
    hello_world();
    return i+j; }
```

La fonction principale `main` est elle-même une fonction qui en appelle d'autres...

Item 4 : Premiers pas avec les fonctions

Les fonctions 8/8 :

- peuvent s'appeler elle-même **récurivement**

```
void recursive_function(int nb_call) {  
    if (nb_call <= 0) return; // stop recursion (otherwise infinite)  
    else {  
        std::cout << "Recursive call nb : " << nb_call << std::endl;  
        recursive_function(--nb_call);  
    }  
}
```

Attention à bien mettre une **condition d'arrêt** dans la fonction réursive...

Item 5 : jouer avec les types

Type constant : il est possible en C++ de définir un type constant :

- `<type> const my_var;`

ou

- `const <type> my_var;`

Une variable de ce type ne pourra alors **pas être modifiée** et doit **être initialisée**.

```
const int i; // ERROR
const int a = 3;
a = 2; // ERROR
float const pi = 3.1416;
pi = 0; // ERROR
```

Item 5 : jouer avec les types

Inférence de type : késako ? 1/4

C'est la possibilité pour le compilateur de déduire le type d'une variable.

Les mot-clés associés sont `auto` et `decltype`

- `auto` vous permet de déclarer une variable sans préciser son type.

```
auto i = 1; // i is deduced to be an int.  
auto f = 5.5f; // F is deduced to be a float.  
auto z; // ERROR !
```

Il est alors **obligatoire d'initialiser les variables.**

Item 5 : jouer avec les types

Inférence de type : késako ? 2/4

- `auto` vous permet de déclarer une variable sans préciser son type.

Ce mot clé apporte trois avantages substantiels

- les variables sont toujours initialisées ;
- plus d'erreur liée à des conversions de type implicite ;
- le code est générique : un changement de type à l'avenir aura moins d'impact.

D'où la règle **AAA** : Almost Always Auto.

Item 5 : jouer avec les types

Inférence de type : késako ? 3/4

- `decltype` vous permet de déterminer le type d'une expression.

```
int i = 1;  
decltype(i) j; // int j;  
int const k = 2;  
decltype(k) l; // int const l;  
decltype(2+3) n; // int n;
```

Item 5 : jouer avec les types

Inférence de type : késako ? 4/4

- `decltype` vous permet de déterminer le type d'une expression.

`decltype`, à la différence d'`auto` déduit exactement le type, y compris :

- son caractère `const` ;
- sa caractéristique mémoire le cas échéant : pointeur `*` ou référence `&` (cf. Items à venir sur la gestion mémoire).

Item 5 : jouer avec les types

Définition d'un **alias de type** : mot clé `using`

On renomme un type `using <alias> = type`

```
using integer = int;  
using      real = double;
```

```
integer i = 0;  
      real a = 1.0;
```

Cela permet :

- d'améliorer la lisibilité du code ;
- d'augmenter la généricité :
 - possibilité de changer un type sous-jacent sans réécrire de code.

Item 5 : jouer avec les types

Introduction aux **espaces des nommages** : `namespace` 1/2

Les `namespace` protègent l'accès aux types afin :

- d'éviter des conflits de noms ;
- d'identifier la provenance des types que l'on utilise.

Syntaxe :

- **déclaration** : `namespace mon_namespace { <bloc de déclarations> }`
- **utilisation** : `mon_namespace::ma_declaration`

Item 5 : jouer avec les types

Introduction aux espaces des nommages : namespace 2/2

Syntaxe :

- déclaration : `namespace mon_namespace { <bloc de déclarations> }`
- utilisation : `mon_namespace::ma_declaration`

```
std::cout << "Hello world"; // namespace std from standard library

namespace my_namespace { // creating a namespace
    int my_int = 0; // declaring a variable inside a namespace
}
int main() {
    my_namespace::my_int +=1; // using a variable from a namespace
}
```

Item 5 : jouer avec les types

Conversion de types : `cast` 1/4

`cast_operator<T>(var)` = conversion dans le type `T` de la variable `var`

Plusieurs opérateurs :

- `static_cast<T>(var)`
- `const_cast<T>(var)`
- `reinterpret_cast<T>(var)`
- `dynamic_cast<T>(var)`

Les types `T` et `decltype(var)` doivent être compatibles...

Item 5 : jouer avec les types

Conversion de types : `cast` 2/4

`cast_operator<T>(var)` = conversion dans le type `T` de la variable `var`

Plusieurs opérateurs :

- `static_cast<T>(var)` : conversion avec vérification à la compilation

```
float pi = 3.14f;  
double pi_dp = static_cast<double>(pi);
```

- `dynamic_cast<T>(var)` : conversion dynamique au runtime.
 - Souvent utilisée pour des conversions entre classes (cf Items Orienté Objet)

Item 5 : jouer avec les types

Conversion de types : `cast` 3/4

`cast_operator<T>(var)` = conversion dans le type `T` de la variable `var`

Plusieurs opérateurs :

- `reinterpret_cast<T>(var)` : opérateur magique ne se soucie pas de la correspondance de type, uniquement d'un accord de tailles entre les 2 types
- `const_cast<T>(var)` : suppression de la composante `const` du type

```
int i = 3;
int const& j = i;
const_cast<int&>(j) = 4; // no more const. now i = 4;
```

Item 5 : jouer avec les types

Conversion de types : `cast` 4/4

Remarque : l'utilisation de `cast` est rarement bon signe...

- C'est un outil qui permet de violer les règles établies...
- C'est souvent (même si pas toujours) un signe de problème de conception...

Item 5 : jouer avec les types

`enum class` : la classe pour les enum...

Pourquoi utiliser des énumérations ? Essayons sans :

```
// Switch between resolution methods: wrong way
int main() {}
    std::string resolution_method{"method1"};

    if (resolution_method == "method1") std::cout << "compute with method1" << std::endl;
    else if (resolution_method == "method2") std::cout << "compute with method2" << std::endl;
    // ...
    else std::cout << "Wrong resolution method." << std::endl;
}
```

- pas robuste (faute de frappe...)
- les différentes méthodes possibles ne sont pas exposées
- risque d'oublier le cas défaut : choix d'une méthode incorrecte.

Item 5 : jouer avec les types

`enum class` : la classe pour les enum...

Pourquoi utiliser des énumérations ? Essayons avec :

```
// Switch between resolution methods: good way
enum class ResolutionMethod {Method1, Method2};
auto resolution_method = ResolutionMethod::Method1;
switch (resolution_method) {
    case ResolutionMethod::Method1: std::cout << "compute with method1" << std::endl;
        break;
    case ResolutionMethod::Method2: std::cout << "compute with method2" << std::endl;
        break;
}
```

- robuste (aide du compilateur, tous les cas doivent être dans le switch)
- les différentes méthodes possibles exposées dans l' `enum`
- le défaut est géré automatiquement

Item 5 : jouer avec les types

`enum class` : la classe pour les enum...

Pourquoi utiliser des énumérations ? Essayons avec :

```
// Switch between resolution methods: good way
enum class ResolutionMethod {Method1, Method2};
auto resolution_method = ResolutionMethod::Method1;
switch (resolution_method) {
    case ResolutionMethod::Method1: std::cout << "compute with method1" << std::endl;
        break;
    case ResolutionMethod::Method2: std::cout << "compute with method2" << std::endl;
        break;
}
```

Exo 5.1 Réalisez un code permettant de choisir entre un affichage des 10 premiers entiers par une boucle for et un affichage par une boucle while. Réalisez deux versions de ce code, l'une avec `if/elseif` et l'autre avec `switch` .

Item 5 : jouer avec les types

`enum class` : la classe pour les enum...

Pourquoi utiliser des énumérations ? Essayons avec :

```
// Switch between resolution methods: good way
enum class ResolutionMethod {Method1, Method2};
auto resolution_method = ResolutionMethod::Method1;
switch (resolution_method) {
    case ResolutionMethod::Method1: std::cout << "compute with method1" << std::endl;
        break;
    case ResolutionMethod::Method2: std::cout << "compute with method2" << std::endl;
        break;
}
```

Pour aller plus loin : évolution des `enum class` dans le standard C++ :

- <https://www.cppstories.com/2024/enum-improvements/>